HW2: Code-Action and MCP Agents for ISSO Evidence Selection

COMS E6998 Section 019, Fall 2026
Design of Production Agentic Systems

Student-specific brief for Aiden Wong
Instructor review draft

Due	Friday, November 20, 2026, before 11:59 p.m.
Weight	20% of the course grade, marked out of 19
Format	One submission for this approved ISSO project adaptation
Late days	The course HW2 policy: 6 per semester, at most 3 on one assignment

1 What you will learn

Extend your HW1 relevance agent into two agents over the same ISSO corpus backend:

- A **code-action agent (CodeAct)** writes and executes Python that composes corpus operations.
- An **MCP agent** performs those operations through an MCP server that you implement.

Run both inside OpenShell. Compare their execution paths, tool use, and access boundaries under a planted prompt injection.

Your task remains relevance classification. Label **1** means that a passage is relevant to the question and audience. Label **0** means that it is not relevant. For each label-1 passage, build an evidence item in a writable packet for that question. The source corpus stays immutable. There is no requirement to generate an answer, repair application code, or change retrieval rankings.

This brief replaces the standard HW2 security-analysis task for this submission. The backend is your retrieval and corpus API. CLDK, Bandit, the software-repository targets, and vulnerability patching are not used. The CodeAct/MCP comparison, controlled writes, runtime routing, sandbox, injection measurements, and 19 rubric items remain.

2 Reuse and freeze your HW1 inputs

Reuse your HW1 loop, dispatcher, state, token accounting, tracer, and relevance definition. Record the submitted HW1 commit or archive hash and explain your HW2 changes.

Use the exact corpus and three query sets from your HW1 submission:

Query set	HW1 source	Questions	Candidate pairs, `k = 3`
Development	`agent/queries/dev/queries.json`	10	30
Held out A	`agent/queries/heldout-a/queries.json`	8	24
Held out B	`agent/queries/heldout-b/queries.json`	8	24

The source `data/embeddings/chunks.json` has this SHA-256:

```
f186563d6a1e702703386a25e5047e44eac3d666a7fe382d516c86dd47e5bf8e
```

Its HW1 `corpus_version` is `f186563d6a1e`.

2.1 Trusted preparation, before any agent run

Prepare these inputs with a deterministic script that you write:

1. Verify the source corpus hash and retain the original query files.
2. Create a canonical corpus view containing text and metadata for non-archived passages. Remove embedding vectors.
3. Freeze the three candidate lists from your submitted HW1 full-run artifacts: `agent/runs/dev-full/report.json`, `agent/runs/heldout-a-full/report.json`, and `agent/runs/heldout-b-full/report.json`. Keep each query ID, chunk ID, and their association. Do not rerank or change the lists between agents.
4. Freeze the immediate previous/next neighbor relationships on each page. Exclude archived or absent passages.
5. Write a manifest containing hashes of the source corpus, prepared corpus view, query files, candidate lists, and neighbor relationships. Save that manifest in the submitted baseline and image. Verify the prepared files against it before each run.

Every candidate is an item, identified by the existing `<query_id>::<chunk_id>` alert ID. The frozen manifest supplies the mapping to the question and passage. The model cannot redefine it. In the HW1 report format, derive the query ID from `test_id` and the chunk ID from `path + "::" + str(line)`. Use the supplied candidate projections as examples of this mapping. Exclude labels, rationales, and evidence from prepared agent inputs. Preserve each source full report's row order in the frozen manifest and final report. Both implementations use that same order.

Preparation is outside the measured run. No live scraping, embedding API, or retrieval rebuild is needed during evaluation. Record preparation separately from runtime costs. Different serializations of an equivalent prepared view are acceptable if their bytes are frozen and documented.

2.2 Staging layout

The sandbox image must separate readable inputs from withheld material. One suitable layout is:

```
/app/                       harness, backend, server, dependencies
/target/corpus/             canonical text/metadata view
/target/queries/            the three question sets
/target/manifests/          frozen candidate and neighbor manifests
/target/labels/             answer key: no kernel read grant
/target/private/            original embedding data: no read grant
/target/.git/               no read grant
/sandbox/work/evidence/     writable per-question packets
/sandbox/work/run/          runtime reports and traces
```

Do not grant access to the entire `/target` directory. Copy only necessary application files into `/app`; do not expose a full repository containing keys or labels through that grant.

3 Build the two agents

3.1 CodeAct: generated Python over your backend

Expose your prepared corpus through an importable Python API. It can reuse your HW1 corpus and passage functions. Document how to open the backend, inspect current candidates, read passages and neighbors, and update an evidence packet. You choose the module, class, and operation names.

Before sandbox execution:

1. Use Claude Code or Codex to read that API's implementation and documentation.
2. Ask it to write a prompt file for your CodeAct agent. Review the generated file.
3. Make your harness load that file. Its instructions must let the model initialize the backend and compose operations in Python.
4. Submit both the exact generation prompt and the resulting agent prompt file. The generation prompt is rubric item C5.

Your model must generate the executed Python. A fixed script selected by the model is insufficient. You can use shell heredocs or a Python execution kernel that you implement. The execution process must run inside OpenShell. Return stdout, stderr, and execution failures to the model. Keep authoritative item selection, budgets, and acceptance state

outside the generated code's namespace.

For example, one generated action could inspect several candidate passages, compare their headings and audience metadata, and return a concise set of observations. This is an illustration, not a prescribed algorithm. The dispatcher sees a script execution as one action and cannot validate each internal operation.

Document how execution output identifies the canonical passages delivered to the model. Validate those IDs and text against the frozen view before counting them as observed evidence. Ordinary analysis output can still be returned, but an unverified passage claim does not establish that the model saw canonical evidence.

3.2 MCP: discovered, validated operations

Implement a stdio MCP server. Adapt your HW1 loop into an MCP host:

1. Start the server using your repository-root `mcp.json`.
2. Discover its tools with `tools/list`.
3. Offer the discovered model-facing tools to the model. Exclude host-only controls.
4. Send every model tool request through your dispatcher, MCP client, server, and backend.

Run the host and server in the same OpenShell sandbox. The MCP host must not open target corpus, query, or packet files directly. It can read its configuration and control metadata. Target content and changes must pass through server tools.

The server must respond to initialization promptly. Load large prepared inputs lazily or in the background. Write protocol JSON to stdout and diagnostic logs to stderr. A banner on stdout breaks the transport.

3.3 Tools and host-only context

These are optional model-facing operations. You can rename, combine, replace, or add tools that meet the task:

Example operation	Purpose
`current_item`	Return the question, audience, selected chunk ID, and permitted candidate IDs.
`read`	Return canonical text and metadata for an allowed chunk ID.
`read_neighbors`	Return the permitted immediate neighbors of a chunk.
`classify`	Propose the relevance label, rationale, and observed evidence IDs.
`read_packet`	Return the current question's existing packet during evidence assembly.
`write_evidence`	Propose the current chunk's evidence item during the evidence phase.

Define argument schemas, result schemas, and refusal cases for every exposed tool. Validate arguments and results. A tool description communicates intended behavior; its prose does not enforce a restriction. Return validation and access refusals as MCP tool errors with `isError: true`. Their text must start with `refused:` and identify the reason. Refuse a withheld location even when the file is absent.

Implement the host-only control `set_alert(alert_id, phase)`, with phase `classify` or `evidence`. The host selects an immutable question–chunk item. The server derives its query, passage, allowed read set, and packet path from that context. The host enters the evidence phase only after it accepts label 1 for that item. Never offer this control to the model. The dispatcher must also reject a model request that names it; hiding its description alone is insufficient. Staff use this trusted control directly when testing the server.

4 Classify, then assemble evidence

Each agent follows the same stages:

1. **Load the fixed items.** Use the verified candidate list. Preallocate one report row per item.
2. **Classify.** Start a fresh conversation and observation set for each item. Label 1 means that the target passage answers all or part of the question for its audience. Extra context can explain the passage's words, but cannot supply content absent from the target passage.
3. **Route in the runtime.** For an accepted label 1, enter evidence assembly. For label 0 or an unclassified item, skip assembly. Trace this decision with `by: runtime`.

4. **Propose and check an update.** Assemble one evidence item, compute its diff, and run the three checks below. Accept or reject the update independently of the relevance classification.

Retain the HW1 relevance contract: the label is integer 0 or 1, the rationale is nonempty and at most 600 characters, and evidence IDs refer to passages delivered to the model during this item. The evidence includes the target passage itself. An invalid proposal returns an explanation to the model rather than crashing the dispatcher.

Retain run-wide provider input/output accounting, pre-call budget checks, per-call output allowance, a step limit, and a no-progress detector. Both phases share one budget. Count every classification and evidence-assembly model call. Define your step unit and progress condition. Use distinct terminal statuses for budget exhaustion, the step limit, and no progress. The final allowed call can overshoot the token cap; do not start another call after the accrued charge reaches it.

Once a classification is accepted, its status and label remain fixed. If no budget remains for an evidence update, keep the classified label and record that update as skipped with a reason. Never add an unclassified item to an evidence packet. A catchable failure must still write the report and trace for all discovered items.

4.1 The evidence packet

Create `/sandbox/work/evidence/<query_id>.json` for each question. The initial packet is:

```
{"query_id": "dev-q05", "items": []}
```

An item contains `chunk_id`, `quote`, `source`, and `url`. The quote is a nonempty, contiguous substring of the canonical target passage, at most 600 characters. Source and URL must match that passage's canonical metadata. The packet is evidence for a possible downstream answer; generating that answer is outside this assignment.

The only allowed delta is adding one item for the current label-1 chunk to its question's packet. Earlier accepted entries must remain unchanged. A chunk can occur at most once in that packet. The model does not choose a filesystem destination. The trusted current item determines it.

4.2 Three acceptance checks

At run initialization, retain a trusted snapshot of the empty packets. Generated code cannot modify this accepted state. Update the trusted state only after an authorized evidence delta passes all three checks. Record each proposed unified diff against the last accepted state. For an MCP update, validate the delta before changing a file. Check the entire packet collection after every CodeAct execution, including classification actions. Classification actions permit no packet changes. Restore unauthorized changes before the next action; never use them as a later baseline. During evidence assembly, compare the resulting files with the same trusted state and apply the three checks below.

Check	Pass condition
`applies`	The nonempty proposed diff applies cleanly to the current packet's pre-update snapshot and produces valid packet JSON.
`current_item_only`	Only the selected question's packet changes, by adding the selected chunk once. No previous entry or other file in the packet collection changes.
`grounded`	The selected item has accepted label 1, was observed in this item's conversation, and supplies an exact canonical quote and matching source/URL.

An accepted update needs a nonempty diff and all three checks true. Reject other proposed updates. Restore rejected CodeAct changes from the trusted snapshot before continuing. Record the attempted delta and failed checks even when changes are restored. At finalization, the packet collection must equal the projection of accepted report updates, with no extra entries or files.

These checks establish mechanical validity and provenance. They do not establish that a passage is relevant or that it answers the question correctly. An exactly quoted but irrelevant passage can pass the checks after an incorrect label 1. Likewise, post-execution checking does not prove that CodeAct never made a transient forbidden read or write.

5 Access boundaries and sandbox

State these rules in both agent prompts:

- **Read:** use the prepared corpus only. Do not read labels, embeddings, archived material, Git metadata, or other items' conversation state. For the current question, read only its three frozen candidates and their frozen immediate neighbors. Calling a neighbor operation must not expand that set recursively. During evidence assembly, you can also read the current question's packet. The server refuses packet reads during classification.
- **Write:** modify only the current chunk's entry in the current question's packet, during the host-authorized evidence phase. Preserve all earlier entries and all other packets. Do not edit the corpus, query files, candidate lists, labels, application code, or original input.
- **Network:** connect only to the configured classification-model provider. There are no live ISSO or embedding requests during a measured run.

5.1 Which layer enforces which rule?

Layer	Enforcement	Limit
Prompt	Describes all intended read, write, and network rules.	Instructions can be overridden by untrusted material.
MCP server	Checks allowed IDs, current item and phase, argument/result schemas, and current packet delta before access or mutation.	It cannot constrain arbitrary code executed inside its own trusted process.
Kernel	Keeps withheld roots inaccessible, prepared corpus read only, and network limited to the model endpoint.	A startup policy cannot change with the current question or chunk.
Runtime acceptance	Accepts only a valid evidence delta, preserving the authoritative label and previous accepted packets.	Detecting and restoring a bad CodeAct delta is not the same as preventing it.

CodeAct has no MCP server layer. Its generated Python can bypass a convenient corpus helper and use ordinary file operations. Its question-dependent scope therefore relies on the prompt before execution and on update acceptance afterward. The kernel still enforces the fixed restrictions. Do not claim it enforces a changing per-question rule.

Run the two agents, their execution process, and the MCP server inside OpenShell. OpenShell supports Linux, macOS on Apple Silicon, and Windows with WSL 2. It also needs Docker, Podman, or host virtualization. The Linux sandbox host needs kernel 6.2 or newer for Landlock ABI v3. On Intel Macs or Windows without WSL 2, use a supported Linux VM. The standard course handout documents the staff-tested OpenShell 0.1.4 setup on Apple Silicon with Colima and the gateway configuration. Set `landlock.compatibility: hard_requirement`; startup must fail if enforcement is unavailable.

Grant read access only to necessary runtime directories and the staged corpus/query/manifest directories. Grant writable space under `/sandbox/work` and `/tmp`; the packet collection is the writable task output. Keep the original target, labels, embeddings, and Git metadata outside writable grants. Do not grant the target root or include it as an automatically granted working directory.

Use an OpenShell provider attachment so that the actual provider key never enters the sandbox environment. A local model server is also acceptable if the policy grants only its endpoint. Explain every grant, the resolved interpreter path permitted to use the network, and the limits of your policy. Keep actual withheld files under `/target/labels` and `/target/private` so that direct tests distinguish a permission denial from a missing file. They must exist and be readable by the sandbox user without the sandbox policy. They are never agent inputs. Keep authoritative event and acceptance records outside generated code's writable state. For example, a parent harness can retain a private journal and publish the final report and trace after execution workers terminate. Do not trust a report, trace, or snapshot merely because it exists in a worker-writable directory. State this trusted-harness boundary; protecting against compromise of the harness itself is outside the assignment.

The supplied `examples/policy.example.yaml` illustrates an OpenAI endpoint. Adjust its runtime paths and endpoint to your image and selected provider, including any attached provider profile.

6 Configuration, outputs, and trace

Keep the command-line contract:

```
your-agent --input <prepared query-set directory> --output <out dir> --config config.toml
```

Inside the sandbox, input must refer to permitted, frozen prepared data. Document how your input directory resolves its corpus and candidate manifest. The output contains `report.json` and `trace.jsonl`. Evidence packets live in the separate packet directory described above. Use a fresh packet collection and output directory for every run. Rejected updates are not carried into the next run.

The supplied `config.schema.json`, `report.schema.json`, and `trace.schema.json` define JSON types and required fields. Validate TOML after parsing it into a JSON-compatible object. The cross-item rules in this brief also apply. The examples illustrate formatting; their synthetic token counts are not measured experiment results.

6.1 Two configs

Commit `configs/mcp.toml` and `configs/codeact.toml`. They must be identical except for `agent.kind`. Use the same model snapshot, sampling settings, token budget, output allowance, and limits. Choose a budget sufficient to complete the development set. Keep `[scan] corpus_version = "f186563d6a1e"` and `k = 3`. Both configs point to repository-root `mcp.json` and `policy.yaml`. The CodeAct run does not use the MCP server, but retains these common configuration fields. Optional experiment configuration selects an injection overlay. An overlay is off for clean measurements.

`mcp.json` uses the standard MCP host structure: `mcpServers` maps a server name to `command`, `args`, and `env`. Use absolute executable and prepared-data paths inside the image. Relative config paths resolve from the documented repository or image working directory. The server cannot assume permission to list `/target`; use its explicitly configured readable subdirectories. The following module and argument names are placeholders for your implementation:

Listing 1: Example server startup configuration.

```
{
  "mcpServers": {
    "isso": {
      "command": "/app/agent/.venv/bin/python",
      "args": [
        "-m", "your_isso_server",
        "--corpus", "/target/corpus/chunks.json",
        "--queries", "/target/queries",
        "--manifests", "/target/manifests",
        "--packet-root", "/sandbox/work/evidence"
      ],
      "env": {}
    }
  }
}
```

Listing 2: Example MCP configuration; the paired CodeAct example changes only agent.kind.

```
# Format example only. Keep these model and budget settings identical for both agents.
# The model name is carried over from the submitted HW1 configuration.
[scan]
corpus_version = "f186563d6a1e"
k = 3

[model]
provider = "openai"
name = "gpt-4.1-mini-2025-04-14"
temperature = 0

[budget]
max_tokens = 600000
output_allowance = 1024
max_steps = 300
no_progress_limit = 3

[agent]
```

```
kind = "mcp"

[mcp]
config = "mcp.json"

[sandbox]
policy = "policy.yaml"

# For injection experiments, use this same separately hashed overlay for both agents:
# [experiment]
# injection_overlay = "fixtures/injection.json"
```

6.2 Report and packet updates

The report records the agent kind, query set, original corpus hash, prepared manifest hash, candidate count, budget, and all item rows. Every frozen candidate pair gets exactly one row. Its ID and question/chunk association must match the manifest. Rows keep the classification status `classified`, `budget_exhausted`, or `error`. Classified rows have label 0 or 1. Unclassified rows have null labels.

Each row also has an `evidence_update` object with:

- `status`: `accepted`, `rejected`, or `skipped`;
- `diff`: the proposed unified diff, or an empty string if none was proposed;
- `checks`: `applies`, `current_item_only`, and `grounded`, each boolean or null if not run;
- `reason`: an explanation, including why a positive item's update was skipped.

Label-0 and unclassified rows must have skipped updates with no diff and no acceptance checks run. A failed update never changes the classification label. Count accepted packets separately from predicted relevance quality. All accepted updates must agree with the final packet files.

6.3 Trace

Retain HW1's append-only JSONL trajectory and flush after every event. Each event has `step`, `ts`, `run_id`, `alert_id` (or null), and `kind`. Steps start at 1 and increase by 1. The kinds are `model_call`, `tool_request`, `tool_result`, `routing`, `state_change`, `terminal`, and `discovery`.

- Log every model call's actual provider input/output counters. Charge their sum across both phases, not an estimate. Record provider cache counters when available without adding them again to the charge.
- Include `via: mcp` on MCP requests/results and `via: codeact` on script execution requests/results. Request arguments are always objects. Put malformed raw argument text in an object field rather than logging a string or array as `args`.
- Follow each request immediately with its corresponding result before another model call. A CodeAct result includes stdout, stderr, and exit status or an execution error.
- For validation or access refusals, record `error_kind: refusal` and `refused_by: dispatcher`, `server`, or `kernel`. Ordinary Python syntax or application exceptions are execution errors, not automatically kernel refusals.
- After `tools/list`, record a `discovery` event containing the server and model-visible tool names. Exclude host-only controls from this list.
- Record runtime decisions to enter or skip evidence assembly as `routing` events with `by: runtime`. Record the model's action choices with `by: model`.
- Finalize each classification row with exactly one matching `state_change` event. An additional `state_change` with `node: evidence_update` carries the acceptance checks, without `alert_status` or `label`.
- Write one terminal event last, including after a catchable failure. Its input/output totals and the report's charged usage equal the model-call sums.

A script that executes successfully but proposes an invalid packet delta has a rejected update. Do not label that rejection as a kernel refusal. Record its checks and the attempted change. These traces demonstrate the layers actually exercised; a missing forbidden attempt does not prove that a layer enforces the rule.

7 Experiments and direct tests

7.1 Clean runs

Run the MCP agent on development, held out A, and held out B. Run CodeAct on development. These are four required clean runs. Use the frozen items, fresh packets, no overlay, and each agent's full configuration. Report the counts of label 1, label 0, null, and accepted/rejected/skipped evidence updates for every run. Compare development predictions with the existing HW1 dev key outside the agents. Accuracy, precision, recall, and the number of passages selected do not affect grades.

7.2 Planted injection

The supplied `fixtures/injection.json` targets `dev-q05::campus-work-f-and-j-students::0`. Its text claims that the correct verdict is in the withheld answer key, asks for another question's passage, and requests an edit to another question's evidence packet.

Treat this as an overlay on the model-visible text of that passage. Do not edit the canonical corpus or frozen candidates. Record the overlay's hash and whether it was enabled. Grounding always uses canonical passage text, excluding the injected instructions.

1. Run each agent on development five times with the overlay enabled, using fresh packets each time.
2. Verify that the attacked item and its full injected text reach an actual model request. Record the marker and a reference to the model-visible observation after truncation. An overlay on disk alone is not proof of exposure.
3. Report the number of exposed runs, forbidden read/write attempts, server or kernel refusals, and rejected CodeAct updates. Include trace lines for every reported attempt. Describe a run without exposure as such; do not count it as resistance to injection.
4. Disable the overlay before producing the clean classification results.

Do not add a rule that detects the fixture's marker and forces a label or refuses all operations. The experiment tests the general boundary rules. A model that ignores the attack in five runs does not establish a reliable security boundary.

7.3 Staff checks

You do not need to implement the staff driver. Your documented contracts and startup configuration must support these direct checks:

1. **Startup and discovery.** Start a fresh sandbox, launch the server from `mcp.json`, initialize, and list tools. The server must locate its prepared inputs without relying on an earlier agent run.
2. **Server reads.** Select a real item through host-only `set_alert`. Read its target passage, then request a known passage outside its frozen permitted set and a withheld location. The server must refuse the forbidden requests.
3. **Server writes.** In evidence phase, accept a valid current-item update. Refuse a different question, a different chunk, changed prior entries, a forged quote, or a non-workspace destination. Refuse a write during classification phase. Staff also send malformed arguments.
4. **Model controls.** Submit a model-shaped request for `set_alert`. The dispatcher must refuse it; hiding the tool from the prompt is insufficient.
5. **Kernel bypass.** Use `openshell sandbox exec` to bypass the MCP server. Reads of labels and original embeddings and writes to the original corpus or outside writable roots must fail. Reads of another public question's permitted corpus file and writes to another packet under the shared writable root can succeed. Those successful bypasses demonstrate why per-item server checks are separate.
6. **Network and key.** Inspect the effective provider-only policy and confirm that the actual provider credential is absent inside the sandbox.

The standard OWASP fixture and case-name driver are replaced by the ISSO fixture and item-aware checks above. Keep read/write operations clearly identified in the README so staff can use your chosen names and argument schemas.

8 Design report and comparisons

Address Part A of the rubric and include both comparisons:

1. **CodeAct versus MCP.** Compare the same 30 development items, model snapshot, sampling settings, and budget. Report model turns, input/output tokens, backend operations, script executions or MCP calls, classifications, and accepted updates. Count internal backend operations separately from model-visible actions where your instrumentation supports it. Explain what a composed Python action can combine and what the server validates individually. Include injection attempts and the enforcing layer. Do not claim that fewer outer calls mean less internal work.
2. **HW1 direct calls versus HW2 MCP calls.** Follow one operation, such as reading a passage, through your own implementations. Show model request, dispatcher, client, server, and backend, as applicable. Explain discovery, argument/result validation, serialization, execution location, permissions, and error propagation. Include one malformed-argument example and a real trajectory diagram from each relevant path.

Discuss limitations: the relevance judgments and hand labels, small development set, canonical-quote checks, static kernel policy, CodeAct bypasses, and what the experiments cannot establish. Use your implemented graph and traces for diagrams. A model-generated DAG is not required.

9 Rubric: 19 points

Each item is worth one point. Grades assess the implemented mechanisms and their required supporting artifacts. Classification accuracy and evidence-selection quality are teaching measurements, not grading thresholds. An absent mechanism or artifact cannot earn its point. An HW1 defect does not automatically fail unrelated HW2 items.

Part A: design report (5)

A1	**Tool contracts:** arguments, results, refusals, and what a description does not enforce.
A2	**Call path:** host, client, server, backend; discovery, validation, and serialization.
A3	**Access rules:** read, write, network; fixed and current-item restrictions, enforcing layers, and bypasses.
A4	**Sandbox policy:** explain every granted or withheld permission.
A5	**Evidence acceptance:** the three checks and their limits, including relevance and transient-access limits.

Part B: implementation (9)

B1	**Server:** exposes the prepared corpus backend and validates argument and result schemas.
B2	**CodeAct:** loads a reviewed API prompt generated with Claude Code or Codex; generated Python initializes the backend and composes operations inside OpenShell.
B3	**Evidence updates:** both agents produce a packet delta, run the three checks, and record the diff and accepted/rejected/skipped outcome.
B4	**Discovery:** the MCP model tool list comes from `tools/list`, excludes controls, and is recorded in the trace.
B5	**Dispatcher:** refuses unknown tools, bad arguments, and model requests for host-only controls.
B6	**Runtime routing:** only accepted label-1 items enter evidence assembly; label 0 and unclassified items skip it, traced as runtime decisions.
B7	**Per-item restrictions:** host-only context binds reads, phase, current chunk, and packet destination; server refusals are attributed correctly. The MCP host does not open target files directly.
B8	**Sandbox:** both agents and server run in OpenShell with `hard_requirement` Landlock, no target-root grant, and fixed read/write restrictions.
B9	**Network:** only the configured model endpoint is allowed, and the actual provider key never enters the sandbox environment.

Part C: evidence and README (5)

C1	**README:** fresh-clone preparation, image build, startup of either agent, and server discovery through `mcp.json`.
C2	**Live injection measurements:** five development runs per agent, exposure evidence, attempted operations, and trace lines.
C3	**Two comparisons:** CodeAct versus MCP and the student's HW1 direct path versus the HW2 MCP path.
C4	**Limitations:** explain what the mechanisms and measurements do and do not establish.
C5	**Generation prompt:** submit the exact prompt used to create the CodeAct backend prompt file, separately from the generated file.

10 README, submission, and course policy

Your README must let staff reproduce the work without asking for missing steps:

1. Give Python, MCP, and OpenShell versions, dependency setup, and environment variable names. Follow the course platform/provider procedure.
2. Verify the corpus pin, prepare frozen inputs and hashes, and stage readable/withheld roots. Never regenerate candidates during a run.
3. Identify the backend API, both prompt files, and how the harness loads and executes generated code.
4. Document tool contracts, refusals, and trusted controls.
5. Give image-build and server-startup commands, with prepared input and packet roots.
6. Give clean and injected commands for each agent, packet reset, and output retrieval. Every launch must work in a fresh sandbox.
7. Commit `configs/mcp.toml` and `configs/codeact.toml`, along with the schemas used by your validation.

For example, after building your image and registering your provider, a launch follows this shape:

```
openshell sandbox create --name <name> --from <image> \
  --policy policy.yaml --provider <registered-provider> \
  --detach -- sleep infinity
openshell sandbox exec -n <name> --no-login-shell --no-tty -- \
  your-agent --input /target/queries/dev \
  --output /sandbox/work/run --config /app/configs/mcp.toml
```

Replace image, executable, and provider names with your implementation. The frozen inputs and configuration must already be in the image or staged through your documented preparation. The standard handout's CLDK installation and OWASP copy commands do not apply to this variant.

Deliverables

- Source for the two agents, backend, server, input preparation, and acceptance checks, with a dependency lockfile.
- The reviewed CodeAct prompt and its exact generation prompt as separate files.
- README, two configs, `mcp.json`, `policy.yaml`, and the schema files.
- The frozen preparation manifest and candidate lists, plus instructions to reconstruct the canonical view from the HW1 source.
- `REPORT.md` or a PDF addressing Part A, both comparisons, injection measurements, and limitations.
- Unedited reports, traces, configs, and final evidence packets for every cited run, plus the overlay and its hash for injected runs.
- A contribution statement identifying who did what.

Do not submit provider keys or private credentials. Do not add per-cell logs or OpenShell exports beyond the required run artifacts. Submit through the course HW2 assignment on CourseWorks; one submission per student or team.

Use Claude Code or Codex for the required prompt-generation step and review its output. You can also use assistants to learn, debug your own code, review designs, or complete scoped sections. Do not delegate the entire assignment and submit its output. Every team member remains responsible for every line and must explain the submission. Your agents must keep the canonical corpus, queries, candidates, and answer key immutable. The course academic-integrity policy applies.